Telematics Platform User Guide

NADIF for 5G ConMod

Document Information	
Filename	TP_F101U-00053_User_Guide_NADIF_ConMod
Status	Release
Revision Date	Jul 7, 2022
Version	02.00
Document ID	TP_F101U-00053

Standard Proprietary Terms
THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2021
Confidential and Proprietary

Table of contents

1 Introduction 4
1.1 Notational Conventions 4
2 Setup 5
2.1 **Tools/Equipment Needed 5**
2.2 **Setup Instructions 5**
2.3 Setup Troubleshooting 6
3 Features/Components 6
3.1 Data Services 6
3.1.1 Overview 6
3.1.1.1 Platform Content Overview 6
3.1.1.2 Product Expectations 6
3.1.1.3 Extending Platform Functionality 7
3.1.2 Setup 7
3.1.2.1 Tools/Equipment Needed 7
3.1.2.2 Setup Instructions 7
3.1.2.3 Setup Troubleshooting 7
3.1.3 Usage 7
3.1.3.1 Basic Instructions 7
3.1.3.2 Detailed Instructions 8
3.1.3.3 Testing/Error Reporting 11
3.2 Voice Call 11
3.2.1.1 Product Expectations 11
3.2.1.2 Extending Platform Functionality 11
3.2.2 Setup 11
3.2.2.1 Tools/Equipment Needed 11
3.2.2.2 Setup Instructions 11
3.2.2.3 Setup Troubleshooting 12
3.2.3 Usage 12
3.2.3.1 Basic Instructions 12
3.2.3.2 Detailed Instructions 12
3.2.3.3 Testing/Error Reporting 13
3.3 External MIPI switch Control 13
3.3.1 Overview 13
3.3.1.1 Platform Content Overview 13
3.3.1.2 Product Expectations 14
3.3.1.3 Extending Platform Functionality 14
3.3.2 Setup 14
3.3.3 Usage 15
3.3.3.1 Basic Instructions 15

© Continental Automotive GmbH 2021
Confidential and Proprietary

3.3.3.2 Detailed Instructions 15
3.3.3.3 Testing/Error Reporting 15
3.4 Modem EFS backup/restore notification handling 16
3.4.1 Overview 16
3.4.1.1 Platform Content Overview 16
3.4.1.2 Product Expectations 16
3.4.1.3 Extending Platform Functionality 16
3.4.2 Setup 16
3.4.2.1 Tools/Equipment Needed 16
3.4.2.2 Setup Instructions 16
3.4.2.3 Setup Troubleshooting 16
3.4.3 Usage 17
3.4.3.1 Basic Instructions 17
3.4.3.2 Detailed Instructions 18
3.4.3.3 Testing/Error Reporting 19
4 Known Issues 20
5 FAQs 20
6 Troubleshooting/Tips and Tricks 20
7 References 20
7.1 Abbreviations/Acronyms/Definitions 20
7.2 Platform Documentation 20
7.3 Website links 20
8 Revision History 21

© Continental Automotive GmbH 2021
Confidential and Proprietary

1 Introduction

This document provides usage information for Network Access Device InterFace, or simply NADIF.

NADIF is defined as a simplified functional interface that allows an application program written in

"C" language to communicate with the embedded cellular device. In addition, NADIF supports multiple clients and provides a simple mechanism for asynchronous event notification via a callback function.

The terms NADIF Server and NADIF Client will be referred to throughout this document according to the following definitions:

- NADIF Server is a Linux daemon that runs on the applications processor and exposes public APIs that can be invoked by user applications to access "service groups" provided by an embedded cellular device (embedded modem, or network access device), such as Voice, Data, SMS, cellular Network, etc. Each "service group" can provide multiple services, for example, voice "service group" can provide services like "dial", "answer", "hang-up" etc.
- NADIF Client is an application program that must be registered in NADIF server to use the APIs.

This user guide will cover details about features provided by these services, configuration usage, and dynamic behavior. The target audience for this document is developers and architects who will use the Telematics Platform NADIF services.

The NADIF Server subsystem resides between the user application and modem layer as shown below:

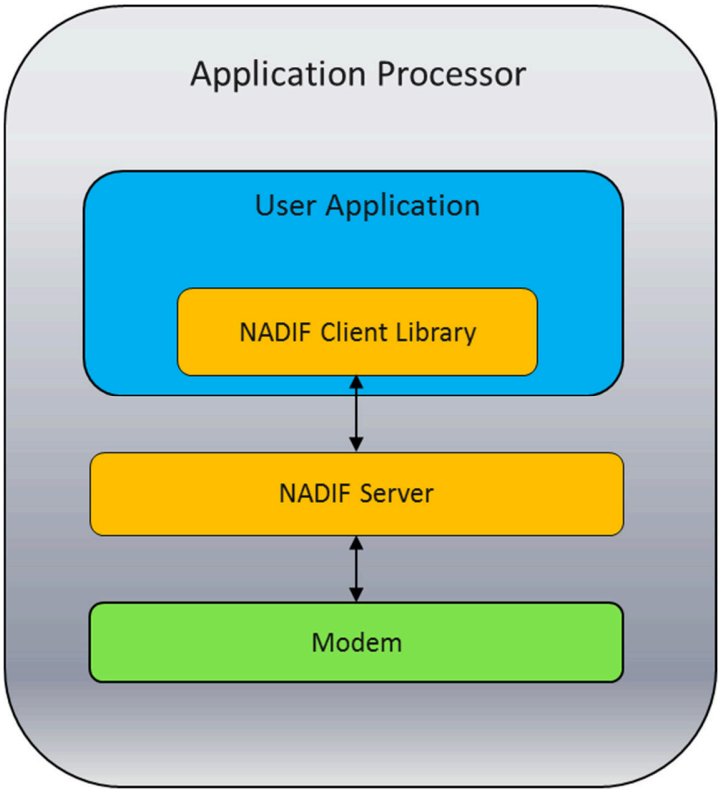


Figure 1: NADIF Context Diagram

1.1 Notational Conventions

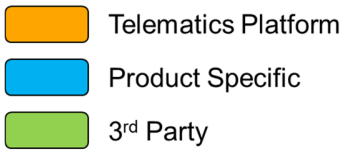
Ctrl+x *A sequence with a '+' indicates that you must press and hold one key or*

© Continental Automotive GmbH 2021
Confidential and Proprietary

	pointing device button while pressing another key or pointing device button
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend



2 Setup

These instructions are designed to help the user of NADIF, also known as NADIF Client, to set up a project capable of using the services provided by NADIF.

2.1 Tools/Equipment Needed

Not Applicable

2.2 Setup Instructions

- The user of NADIF must use the *NADIF Client Library,* this library is responsible for controlling all socket logic, such as connection, sending request, and listening to notifications and asynchronous response.
- The *NADIF Client Library* is coined as **conti-nadif.** The user of NADIF has to use pkg-config in the client application Makefile to link to the NADIF Client library.
```
LDFLAGS += `pkg-config --libs conti-nadif`
CFLAGS += `pkg-config --cflags conti-nadif`
```
- NADIF Client must include **nadif.h** and **nadif_control.h** in the source file to access common defines used by NADIF.
```
#include "nadif.h"
#include "nadif_control.h"
```
- NADIF Client must be registered in NADIF Server to invoke any NADIF API and receive notifications, see all details in Client Registration.
- In order to invoke any API, NADIF Client must include the appropriate header file according to the service group that the API belongs to. See the Product Expectations subsection of each API service group to find the appropriate header file.
- **Security**: Usage of NADIF APIs require clients (user) to be a member of '**modem**' group (see TP_F101U-00009).

© Continental Automotive GmbH 2021
Confidential and Proprietary

2.3 Setup Troubleshooting

Not Applicable

3 Features/Components

3.1 Data Services

3.1.1 Overview

3.1.1.1 Platform Content Overview

This section describes the support provided by NADIF regarding the APIs related to data session.

See the following features supported by NADIF:

- Start a data session.
- Stop a data session.
- Set Packet Data Configuration.
- Get Packet Data Configuration.
- Get the status of a data session.
- Get the status of a IMS services.
- Get the SIP URI of IMS Session.
- Get the PDN throttle information.

3.1.1.2 Product Expectations

NADIF Client must include the header file `notification.h` to access any API described in this section.

3.1.1.2.1 Start Data Session

The following expectations must be met by NADIF Client in order to start a data session:

1. The modem must be registered on the cellular network.
2. NADIF Client is expected to use `NADIF_data_start()` to start a data session.

3.1.1.2.2 Stop Data Session

NADIF Client is expected to use `NADIF_data_stop()` to stop a data session.

3.1.1.2.3 Set Packet Data Configuration

NADIF Client is expected to use `NADIF_data_set_pd_configuration()` to configure the parameters required to connect to the packet data network.

3.1.1.2.4 Get Packet Data Configuration

NADIF Client is expected to use `NADIF_data_get_pd_configuration()` to get the configuration of the current packet data network.

3.1.1.2.5 Get Data Session Status

NADIF Client is expected to use `NADIF_data_get_status()` to get the status of a data session.

3.1.1.2.6 Get IMS Services Status

NADIF Client is expected to use `NADIF_data_get_ims_status()` to query information about the status of the IP multimedia subsystem and its services.

3.1.1.2.7 Get IMS Session SIP URI

© Continental Automotive GmbH 2021
Confidential and Proprietary

NADIF Client is expected to use `NADIF_ims_get_sip_uri()` to query the SIP URI of IMS Session.

3.1.1.2.8 Get PDN Throttle Information

NADIF Client is expected to use `NADIF_data_get_pdn_throttle_info()` to get the PDN throttle information for the current packet data connections.

3.1.1.3 Extending Platform Functionality

Not applicable

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Not applicable

3.1.2.2 Setup Instructions

Not applicable

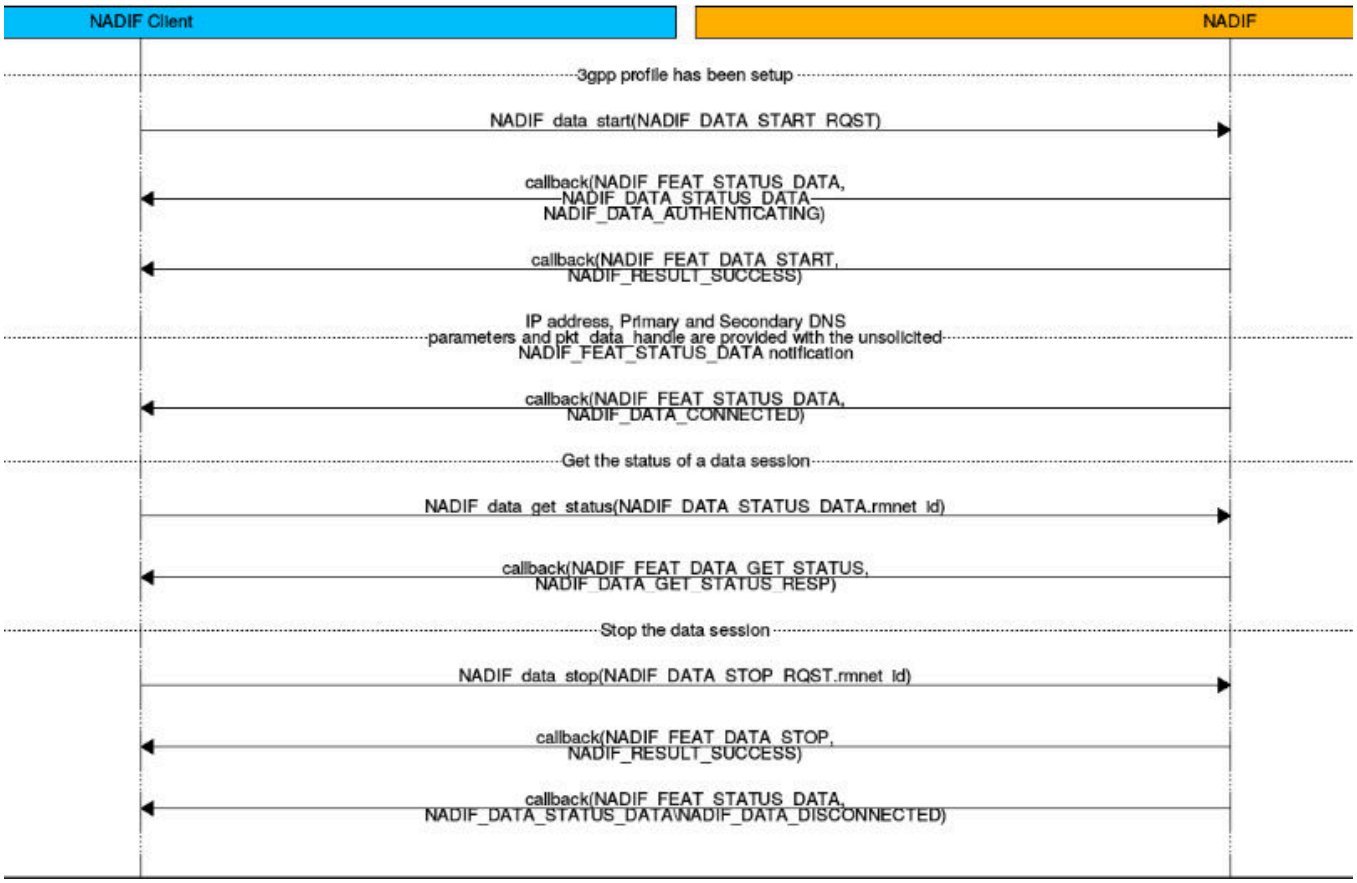
3.1.2.3 Setup Troubleshooting

Not applicable

3.1.3 Usage

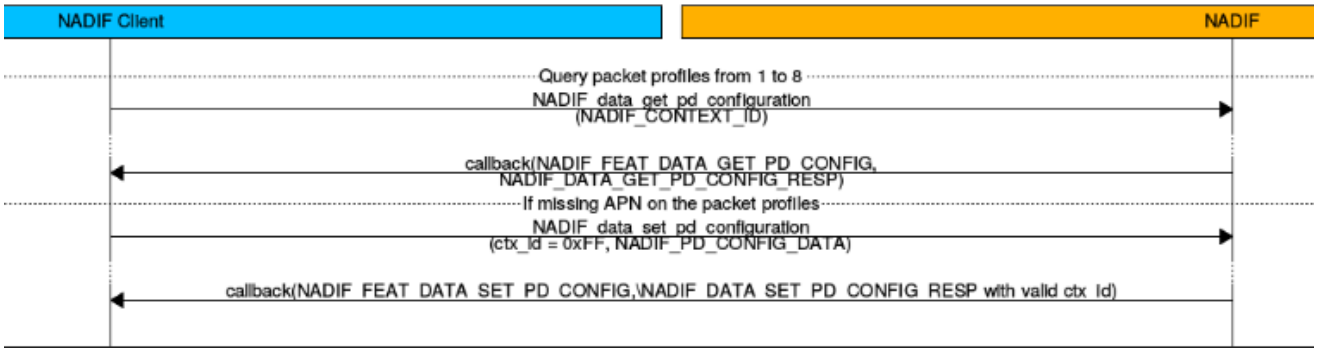
3.1.3.1 Basic Instructions

MSC illustrating the overview of the Data session:



© Continental Automotive GmbH 2021
Confidential and Proprietary

MSC illustrating Packet Data configuration API:



3.1.3.2 Detailed Instructions

3.1.3.2.1 Start data session

API/Type Description	Doxygen Path

Structure of data start request	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Data Services`
	`|-> NADIF_DATA_START_RQST`

API to start a data session	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Data Services`
	`|-> NADIF_data_start()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_RESULT`

This API allows NADIF Client to start a data session. Note the notification `NADIF_RESULT` is reported with `feat_id` set to `NADIF_FEAT_DATA_START` when the data session is started.

3.1.3.2.2 Stop data session

API/Type Description	Doxygen Path

Structure of data stop request	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Data Services`
	`|-> NADIF_DATA_STOP_RQST`

API to stop a data session	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Data Services`
	`|-> NADIF_data_stop()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`

© Continental Automotive GmbH 2021
Confidential and Proprietary

API/Type Description	Doxygen Path
	|-> **NADIF_RESULT**

This API allows NADIF Client to stop a data session. Note the notification `NADIF_RESULT` is reported with `feat_id` set to `NADIF_FEAT_DATA_STOP` when the data session is stopped.

3.1.3.2.3 Set Packet Data Configuration

API/Type Description	Doxygen Path
Structure of Packet Data configuration request data	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_PD_CONFIG_DATA**
API to set Packet Data configuration	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_data_set_pd_configuration()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_DATA_SET_PD_CONFIG_RESP**

This API allows NADIF Client to configure the parameters required to connect to the packet data network. Note the notification `NADIF_DATA_SET_PD_CONFIG_RESP` is reported with `feat_id` set to `NADIF_FEAT_DATA_SET_PD_CONFIG` in response to the request.

3.1.3.2.4 Get Packet Data Configuration

API/Type Description	Doxygen Path
Structure of Packet Data configuration request data	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_PD_CONFIG_DATA**
API to get Packet Data Configuration	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_data_get_pd_configuration()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_DATA_GET_PD_CONFIG_RESP**

This API allows NADIF Client to get the configuration of the current packet data network. Note the notification `NADIF_DATA_GET_PD_CONFIG_RESP` with `feat_id` set to `NADIF_FEAT_DATA_GET_PD_CONFIG` provides packet data network configuration in response to the request.

3.1.3.2.5 Get Data Session Status

API/Type Description	Doxygen Path

© Continental Automotive GmbH 2021
Confidential and Proprietary

API/Type Description	Doxygen Path
Structure of Packet Data status request data	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_DATA_STATUS_DATA**
API to get the status of a data session	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_data_get_status()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_DATA_GET_STATUS_RESP**

This API allows NADIF Client to get the status of a data session. Note the notification `NADIF_DATA_GET_STATUS_RESP` with `feat_id` set to `NADIF_FEAT_DATA_GET_STATUS` provides data session status in response to the request.

3.1.3.2.6 Get IMS Services Status

API/Type Description	Doxygen Path
Structure of IMS Service status request data	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_GET_IMS_STATUS_RQST**
API to get the status of a IMS services	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_data_get_ims_status()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_GET_IMS_STATUS_RESP**

This API allows NADIF Client to query information about the status of the IP multimedia subsystem and its services. Note the notification `NADIF_GET_IMS_STATUS_RESP` with `feat_id` set to `NADIF_FEAT_DATA_GET_IMS_STATUS` provides IMS service status in response to the request.

3.1.3.2.7 Get IMS Session SIP URI

API/Type Description	Doxygen Path
API to get the SIP URI of IMS session	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_ims_get_sip_uri()**
Notification data type of API	Modules |-> Modem |-> NADIF

© Continental Automotive GmbH 2021
Confidential and Proprietary

API/Type Description	Doxygen Path
	`|-> Data Services` `|-> NADIF_IMS_GET_SIP_URI_RESP`

This API allows NADIF Client to query the SIP URI of IMS Session. Note the notification `NADIF_IMS_GET_SIP_URI_RESP` with `feat_id` set to `NADIF_FEAT_IMS_GET_SIP_URI` provides IMS Session SIP URI in response to the request.

3.1.3.2.8 Get PDN Throttle Information

API/Type Description	Doxygen Path

API to get the PDN throttle information	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Data Services`
	`|-> NADIF_data_get_pdn_throttle_info()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Data Services`
	`|-> NADIF_DATA_GET_PDN_THROTTLE_INFO_RESP`

This API allows NADIF Client to get the PDN throttle information of the current packet data connections. Note the notification `NADIF_DATA_GET_PDN_THROTTLE_INFO_RESP` with `feat_id` set to `NADIF_FEAT_DATA_GET_PDN_THROTTLE_INFO` provides PDN throttle information in response to the request.

3.1.3.3 Testing/Error Reporting

Not applicable

3.2 Voice Call

3.2.1.1 Product Expectations

NADIF Client must include the header file `nadif_voice.h` to access any API described in this section.

3.2.1.1.1 Custom user data re-transmission using SIP over IMS voice call

The following expectations must be met by NADIF Client in order to update custom user data re-transmission using SIP over IMS voice call as designed:

1. The modem must be registered on the cellular network with IMS support.
2. NADIF Client is expected to use `NADIF_voice_update_custom_data_over_ims()` to update custom user data re-transmission using SIP over IMS voice call.

3.2.1.2 Extending Platform Functionality

Not applicable

3.2.2 Setup

3.2.2.1 Tools/Equipment Needed

Not applicable

3.2.2.2 Setup Instructions

Not applicable

© Continental Automotive GmbH 2021
Confidential and Proprietary

3.2.2.3 Setup Troubleshooting

Not applicable

3.2.3 Usage

3.2.3.1 Basic Instructions

Not applicable

3.2.3.2 Detailed Instructions

3.2.3.2.1 Custom user data re-transmission using SIP over IMS voice call

API/Type Description	*Doxygen Path*

API to dial	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Voice`
	`|-> NADIF_voice_update_custom_data_over_ims()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_RESULT`

This API allows NADIF Client to provide the updated custom user data re-transmission using SIP over IMS voice call whenever the unsolicited notification NADIF_FEAT_REQ_IMS_CUSTOM_DATA_UPDATE is received.

Notification NADIF_FEAT_VOICE_UPDATE_CUST_USER_DATA_TRANSMISSION indicates the custom user date update result status.

Note: The notification NADIF_FEAT_REQ_IMS_CUSTOM_DATA_UPDATE is received whenever updated custom user data is requested by the network.

© Continental Automotive GmbH 2021
Confidential and Proprietary

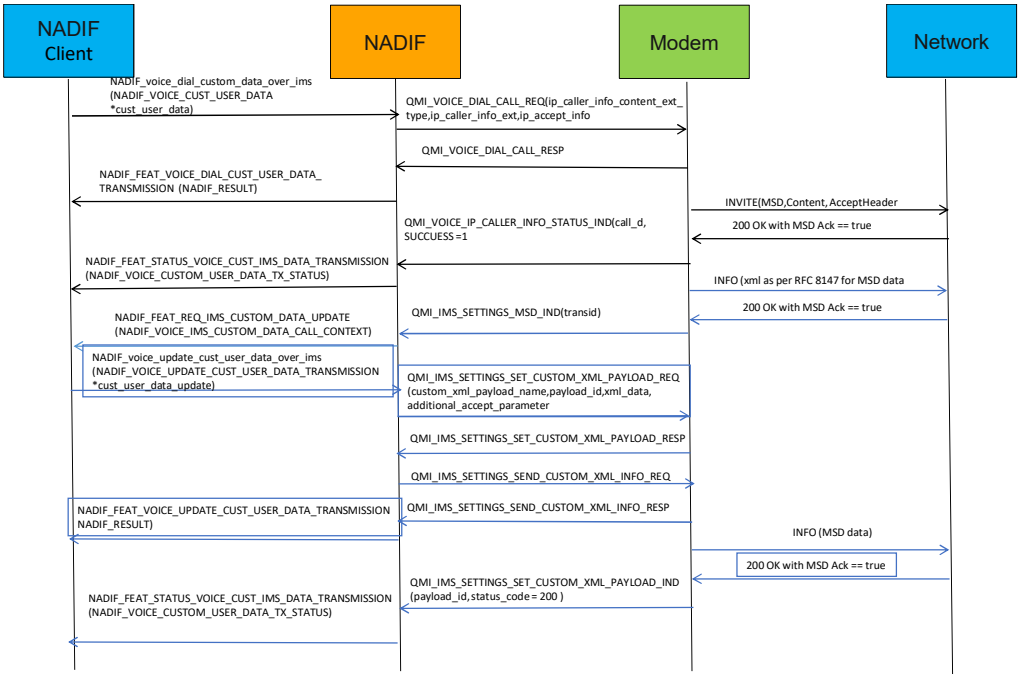


Figure 2: Custom user data re-transmission using SIP over IMS voice call

3.2.3.3 Testing/Error Reporting

Not applicable

3.3 External MIPI switch Control

3.3.1 Overview

3.3.1.1 Platform Content Overview

This section describes the support provided by NADIF to request the switch position of MIPI compliant RF switches that are external to the NAD.

- Switch(es) must be on the RFFE0 bus.
- The switches are only written to
  - On startup
  - In response to the API call
  - Exiting airplane mode
- No power management of the switches is provided (see 3.3.1.2 Product Expectations)
- No capability to query or otherwise confirm the state of the switch is provided.

The supported switch positions are switch and product specific and are defined in RF driver compiled in the platform release.

Due to the confidentialy arrangements with each switch vendor the platform will prepare a specific App Note for each switch and product configuration. That App Note will be required to use this functionality in NADIF to populate the fields described in the ICD.

3.3.1.1.1 Initial Switch Position

© Continental Automotive GmbH 2021
Confidential and Proprietary

The modem will set the switch(es) to the default position described in the App Note at the following events:

1. Modem power-up
2. Exit from Airplane Mode

On power on or entering airplane mode, most RF switches are in full isolation mode, meaning that there are no antennas connected to the NAD's RF ports. This functionality is provided to ensure that the switch is taken out of isolation as early as possible in the boot process and that there is no dependency on the client to allow the modem to start searching for a network.

The same behavior when exiting Airplane Mode is a side effect of the underlying modem vendor's implementation.

3.3.1.2 Product Expectations

NADIF Client must include the header file `nadif_modem.h` to access any API described in this section.

3.3.1.2.1 Initial State

The client does not need to take any action to set the state of the switch in the use cases described in 3.3.1.1.1 Initial Switch Position if the default state is acceptable for use.

If the client wants to set the state of the antenna at these events, they must ensure that the command does not conflict with the autonomous behavior (e.g. insert some delay in the units of seconds between the event and the API call) and observe all other requirements in 0 Switch Position Request

3.3.1.2.2 Dynamic Position

Switch positions can only be changed outside of a cellular call (packet data or voice). The underlying modem vendor's behavior is not guaranteed in the case of the API being called while in a cellular call.

The underlying modem vendor does not provide any mechanism to positively confirm the state of the switch. The API only provides the feedback that the command was successfully accepted by the vendor's subsystem.

As the behavior is dependent on the arrangement of the switch(es) a single solution cannot be recommended. Instead, the client is recommended to test and evaluate the behavior of the system in response to switch position changes in both sunny day (all antennas present) and fault (varying number of antennas missing) conditions across different Radio Access Technologies on each specific product configuration. A single general solution should be achievable unless minimized KPIs are required for specific scenarios.

Instead, the client Client is recommended to test and evaluate the behavior of the system in response to switch position changes in both sunny day (all antennas present) and fault (varying number of antennas missing) conditions across different Radio Access Technologies on each specific product configuration. A single general solution should be achievable unless minimized KPIs are required for specific scenarios.

It is recommended that the retry strategy include a maximum timeout for registration after a switch position change, after which the client will proceed with the next call to provide a failsafe against the case that a registration event is not received by the client but the system is capable of connecting a call.

3.3.1.2.3 Power Management of switch(es)

No power management of the external switches is provided.

The client must be aware of cases where the power is removed from and re-applied to the swtich or the state of switch is otherwise taken out of normal (full) operation mode. If these cases are not followed by the events described in 3.3.1.1.1 Initial Switch Position within the necessary time to meet system requirements, the client must use the API to guarantee the state of each switch.

3.3.1.3 Extending Platform Functionality

Not applicable

3.3.2 Setup

Not applicable

© Continental Automotive GmbH 2021
Confidential and Proprietary

3.3.3 Usage

3.3.3.1 Basic Instructions

Not applicable

3.3.3.2 Detailed Instructions

Switch Position Request

API/Type Description	Doxygen Path

API to set MIPI switch position	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Modem Status`
	`|-> NADIF_modem_set_mipi_switch_registers ()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Modem Status`
	`|-> NADIF_FEAT_SET_MIPI_SWITCH_REGISTER`

The switch position should only be changed while not in a cellular call (voice or packet data). Behavior and response time are undefined if the command is sent while in a call.

An error can be treated as a positive indication that the the switch position did not change. At least one retry is recommended in the case of an error scenario.

Since there is no feedback from the switch position, positive confirmation of the switch change is not possible. The switch change is expected to occur within 10ms of the command being issued, but it is recommended to enforce some minimum delay (e.g. 1s) and/or wait for the modem to re-register with the new antenna configuration before triggering the next cellular call.

3.3.3.3 Testing/Error Reporting

Error Condition	Possible cause	Corrective Action
When trying to set the MIPI switch calling NADIF_modem_set_mipi_switch_registers()API returns NADIF_RESULT_FAILURE.	Modem cannot communicate with MIPI switch	1. Check if correct RFC has been flashed 2. Check physically if the MIPI switch is correctly connected 3. Check if correct hardware version with physical MIPI switch is being used
Modem does not receive signal at power up.	MIPI switch has not been initiated properly or has been initiated in isolation state.	1. Check correct RFC has been flashed 2. Check physically if the MIPI switch is correctly connected 3. If MIPI switch hardware is updated. A proper driver should be added to the modem software.

© Continental Automotive GmbH 2021
Confidential and Proprietary

3.4 Modem EFS backup/restore notification handling

3.4.1 Overview

3.4.1.1 Platform Content Overview

This section describes the support provided by NADIF for EFS backup/restore functionality.

Modem EFS (Embedded file system) partition corruptions typically causes the NAD to be unusable for any connectivity related operations. To recover from this an automatic EFS backup and recovery mechanism is implemented which is working as expected. In some unforeseen cases, there is a chance of EFS restore failure. If there is no success in EFS Restore, the EFS restore operation will be performed in loop until EFS is recovered. To limit such cyclical loop, mechanism is implemented to perform EFS Restore from Runtime backed up EFS data for 3 attempt and if any failure seen from EFS restore, then EFS Restore will be done from Factory backup data. Even if it fails with 3 attempts with Factory backed up EFS data, this unsolicited notification will indicate EFS Recovery Exhaust process.

3.4.1.2 Product Expectations

SDK version 3.3.146.0 or newer

Product application shall consider additional 8MB of RAM during EFS backup.

Product application shall never modify or delete “efs_backup.bin” and “factory_backup.bin”.

Product application shall keep NAD power ON during the EFS Backup/Restore.

Product application shall trigger factory backup using NADIF API. This backup would be Golden copy of Calibration information which acts as a last resort of recovery in case if runtime backup’s unable to recover the modem.

Product application shall trigger factory backup to get the Golden copy of Calibration information file after the device was first calibrated in the factory.

Product application shall subscribe to modem status notifications using NADIF_init (Table 3.4.3.2.1).

3.4.1.3 Extending Platform Functionality

Not applicable

3.4.2 Setup

3.4.2.1 Tools/Equipment Needed

Not applicable

3.4.2.2 Setup Instructions

Platform is introducing a way to automatically do periodic backup (every 15 days - Hardcoded by Modem Vendor) of the Modem EFS data on Linux file system. The periodic timer can be updated through persistence key “PeriodicEFSBackupTimer”.

EFS backup location is “/data/efs_backup/”, which is created by the first backup.

3.4.2.3 Setup Troubleshooting

Not applicable

© Continental Automotive GmbH 2021
Confidential and Proprietary

3.4.3 Usage

3.4.3.1 Basic Instructions

API/Type Description	Doxygen Path
API to trigger modem EFS factory backup generation. This function creates EFS factory backup file "factory_backup.bin" at EFS backup location.	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_internal_trigger_factory_efs_backup()**
Event notification for EFS Backup/Restore modem status	Modules |-> Modem |-> NADIF |-> Modem Status |-> ***NADIF_FEAT_STATUS_MODEM_EFS_IND***
API to get the last known reason(s) for resuming from suspended state	Modules |-> Hardware Abstraction Layer (HAL) |-> HAL Library |-> System Interface |-> **tpHAL_getLastResumeReason**

Full list of supported resume reasons for HAL API **tpHAL_getLastResumeReason** is available at Power management user Guide (see **TP_F101U_00039**):

© Continental Automotive GmbH 2021
Confidential and Proprietary

3.4.3.2 Detailed Instructions

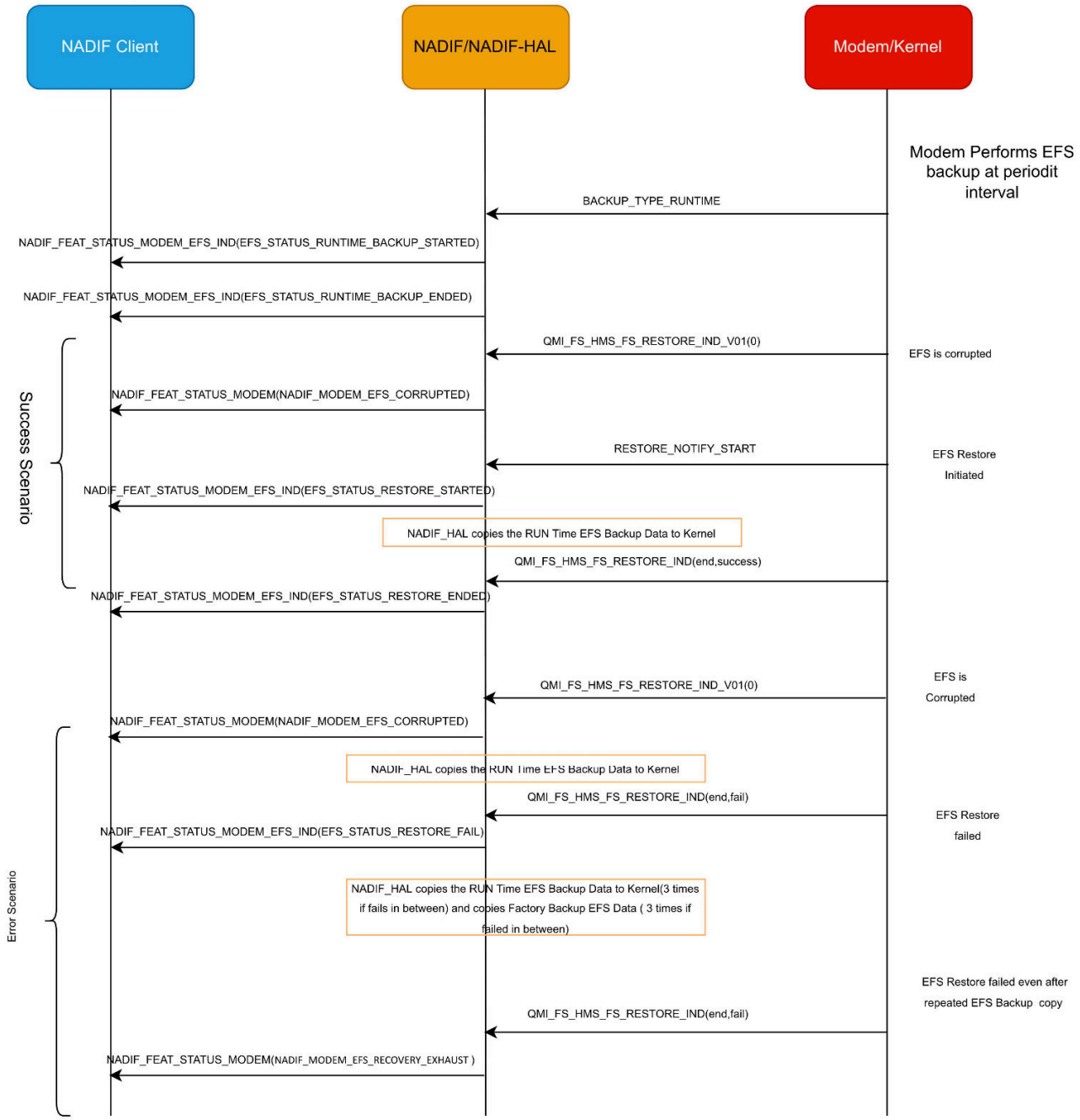


Figure 17. Handling runtime backup and restore modem notifications sequence diagram

© Continental Automotive GmbH 2021
Confidential and Proprietary

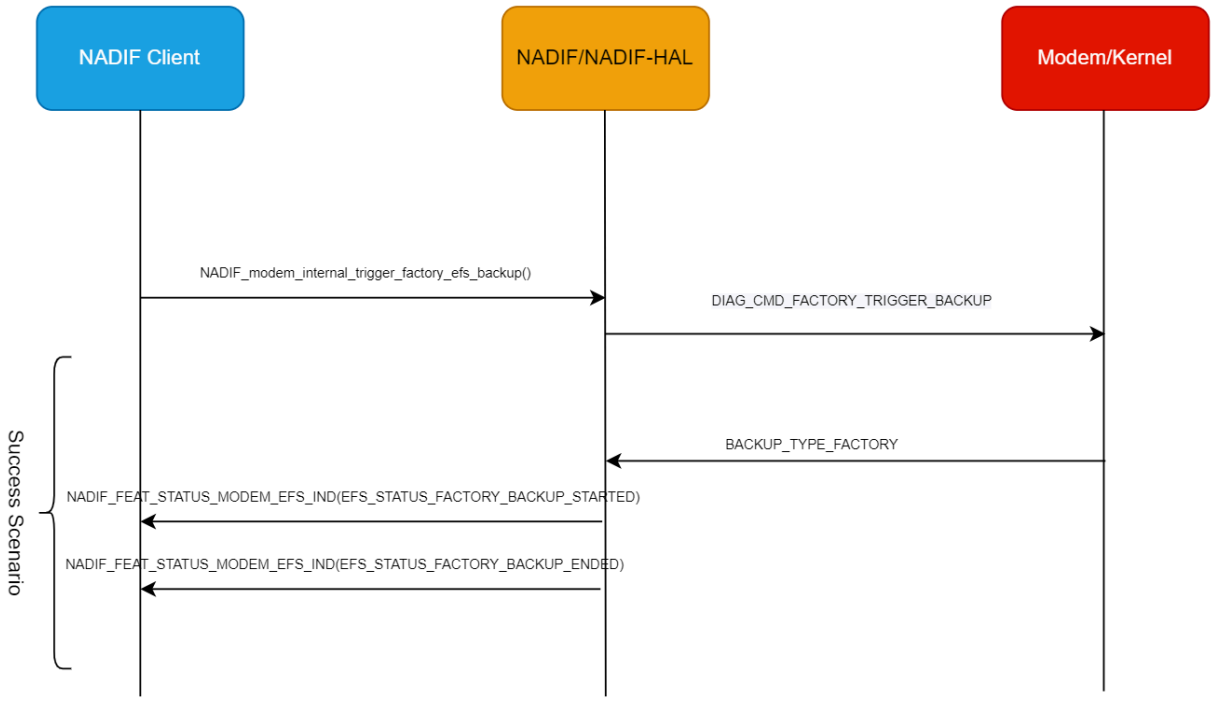


Figure 18. Creating factory backup sequence diagram

3.4.3.2.1 Pseudo code example to get change of modem EFS status

```
uint8 callback(NOTIFICATION_REQUEST_DATA *rqst_data, uint32 size, void *data) {
   switch(rqst_data->feat_id) {
      case NADIF_FEAT_STATUS_MODEM_EFS_IND:
         modem_efs_status_ind = (NADIF_MODEM_EFS_STATUS_INFO *)data;
         Log("EFS Status.:%d",*modem_efs_status_ind);
         break;
   }
}
NADIF_init(callback, &nadif_init_client_id);
```

3.4.3.3 Testing/Error Reporting

Not applicable

© Continental Automotive GmbH 2021
Confidential and Proprietary

4 Known Issues

Not Applicable

5 FAQs

Not Applicable

.

6 Troubleshooting/Tips and Tricks

Not Applicable

7 References

7.1 Abbreviations/Acronyms/Definitions

Not applicable

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary
TP_F101U-00009: Telematics Common Service User Guide
TP_F101U-00023: Common Library User Guide
TP_F101U-00024: Connection Manager User Guide
TP_F101U-00026: Persistence User Guide
TP_F101U-00036: NADIF 5G NAD User Guide
TP_F101U-00039: Power Management User Guide

7.3 Website links

3GPP specifications: http://www.etsi.org

3GPP TS 24.008, release 12:
https://www.etsi.org/deliver/etsi_ts/124000_124099/124008/12.15.00_60/ts_124008v121500p.pdf

3GPP TS 24.301, release 12:
https://www.etsi.org/deliver/etsi_ts/124300_124399/124301/12.13.00_60/ts_124301v121300p.pdf

3GPP TS 27.005, release 12:
https://www.etsi.org/deliver/etsi_ts/127000_127099/127005/12.00.00_60/ts_127005v120000p.pdf

3GPP TS 27.007, release 12:
https://www.etsi.org/deliver/etsi_ts/127000_127099/127007/12.12.00_60/ts_127007v121200p.pdf

© Continental Automotive GmbH 2021
Confidential and Proprietary

8 Revision History

Version	Date	Description
01.00	Sep 15, 2021	Initial release
02.00	Jul 07, 2022	Add chapter 3.4 “Modem EFS backup/restore notification handling”

© Continental Automotive GmbH 2021
Confidential and Proprietary